CSCI6032 lecture 19 20 Logistic Regression

Reading

Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press, 2016). Free online at **deeplearningbook.org**.

Read these sections before and alongside lectures 19–20:

- **§5.5 Maximum Likelihood Estimation**, including §5.5.1 Conditional Log-Likelihood and Mean Squared Error and §5.5.2 Properties of Maximum Likelihood.
- **§5.7.1 Probabilistic Supervised Learning.** Introduces logistic regression,
  $p(y = 1 \mid x; \theta) = \sigma(\theta^T x)$,
  and explains why, unlike linear regression, it has no closed-form solution and is fit by maximizing the log-likelihood with gradient descent.
- **§6.2.1.1 Learning Conditional Distributions with Maximum Likelihood.** Why minimizing cross-entropy loss is the same as maximum likelihood estimation.
- **§6.2.2.2 Sigmoid Units for Bernoulli Output Distributions.** Logistic regression viewed as the output unit of a neural network, and why the log-likelihood loss avoids the sigmoid's saturation.
- **§3.10 Useful Properties of Common Functions.** The logistic sigmoid and softplus identities used when differentiating the log-likelihood.

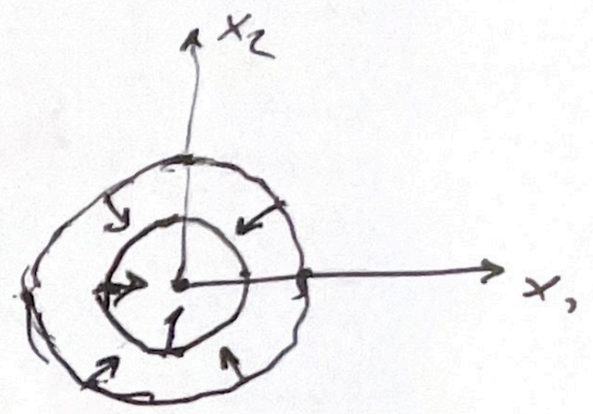
Gradients

$$\nabla f(\bar{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

Ex: Paraboloid

$$f(x) = -(x_1^2 + x_2^2)$$

$$\nabla f(\bar{x}) = \begin{bmatrix} -2x_1 \\ -2x_2 \end{bmatrix}$$



Gradient points towards steepest ascent.

Trace operator

$$tr(A) = \sum_{i=1}^{n} A_{ii}$$

Ex: $A = \begin{bmatrix} 1 & 3 \\ 2 & 7 \end{bmatrix}$

$tr(A) = 1+7 = 8$

Traces are linear

$tr(A+B) = tr(A) + tr(B)$

$tr(cA) = c\ tr(A)$

Ex: $A = \begin{bmatrix} 1 & 3 \\ 2 & 7 \end{bmatrix}$, $B = \begin{bmatrix} 4 & 0 \\ 1 & 1 \end{bmatrix}$

$tr(A) = 1+7$, $tr(B) = 4+1 = 5$, $tr(A)+tr(B) = 13$

$tr(A+B) = tr\begin{bmatrix} 5 & 3 \\ 3 & 8 \end{bmatrix} = 13$

Trace (cont.)

Traces are cyclic

$$tr(ABC) = tr(BCA) = \overbrace{tr(CAB)}$$

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix}, \quad B = \begin{bmatrix} 4 & 0 \\ 1 & 1 \end{bmatrix}, \quad C = \begin{bmatrix} 2 & \frac{1}{2} \\ 2 & 1 \end{bmatrix}$$

$$ABC = \begin{bmatrix} 6 & 2 \\ 19 & 7 \end{bmatrix} \begin{bmatrix} 2 & \frac{1}{2} \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 16 & 5 \\ 52 & 16\frac{1}{2} \end{bmatrix}$$

$$tr(ABC) = 32\frac{1}{2}$$

Rotate left one

$$tr(BCA) = tr\begin{bmatrix} 14 & 30 \\ 8\frac{1}{2} & 18\frac{1}{2} \end{bmatrix} = 14 + 18\frac{1}{2} = 32\frac{1}{2}$$

Jacobians

$$\vec{f}(\vec{x}) = \begin{bmatrix} f_1(\vec{x}) \\ f_2(\vec{x}) \\ \vdots \\ f_n(\vec{x}) \end{bmatrix}$$

$\vec{f}$ is a vector function.

f_i is the ith component of the vector function

f_i is a scalar function

$$J_f = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & & \frac{\partial f_2}{\partial x_n} \\ \vdots & & \ddots & \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}$$

Jacobians (cont.)

It is common in machine learning, statistics, and economics to use nabla ∇ to denote the Jacobian when it is applied to a vector function.

$$\nabla f(\vec{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & & \ddots & \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}$$

$f(\bar{x}) = A\bar{x}$ ← This is a vector function

A is square matrix

$\bar{x}$ is a column vector

$$A = \begin{bmatrix} 3 & 5 \\ 2 & 4 \end{bmatrix}, \quad \bar{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$A\bar{x} = \begin{bmatrix} 3x_1 + 5x_2 \\ 2x_1 + 4x_2 \end{bmatrix} \begin{matrix} \leftarrow x_1 \text{ component of the vector} = f_1 \\ \leftarrow x_2 \text{ component of the vector} = f_2 \end{matrix}$$

$$\nabla_x A\bar{x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1}(3x_1 + 5x_2) & \frac{\partial}{\partial x_2}(3x_1 + 5x_2) \\ \frac{\partial}{\partial x_1}(2x_1 + 4x_2) & \frac{\partial}{\partial x_2}(2x_1 + 4x_2) \end{bmatrix}$$

$$= \begin{bmatrix} 3 & 5 \\ 2 & 4 \end{bmatrix}$$

$$\nabla_x A\bar{x} = A$$

This is similar to $\frac{d}{dx} ax = a$ when dealing with one-dimensional scalar functions.

Gradient (derivative) with respect to a matrix

A & X are $n \times n$ matrices.



$$\nabla_X AX = ?$$

So far we have used ∇ to denote the gradient of scalar function and to denote the Jacobian of a vector function.

It is hard to visualize what this could mean. If we want to apply a gradient operation then we must somehow reduce the matrix to a scalar function. We must scalarize X.

We do this as part of the definition of applying the gradient with respect to a matrix.

Known as the trace trick.

$$\nabla_X \operatorname{tr}(AX)$$

gradient

The trace function "scalarizes" X.

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad X = \begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{bmatrix}$$

$$AX = \begin{bmatrix} 1 \cdot x_{11} + 2 \cdot x_{21} & 1 \cdot x_{12} + 2 \cdot x_{22} \\ 3 \cdot x_{11} + 4 \cdot x_{21} & 3 \cdot x_{12} + 4 \cdot x_{22} \end{bmatrix}$$

$\operatorname{tr}(AX) = (x_{11} + 2x_{21}) + (3x_{12} + 4x_{22}) = f$ ← This is now a scalar function!

$$\nabla_X = \begin{bmatrix} \frac{\partial f}{\partial x_{11}} & \frac{\partial f}{\partial x_{12}} \\ \frac{\partial f}{\partial x_{21}} & \frac{\partial f}{\partial x_{22}} \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

$\boxed{\nabla_X AX = A^T}$ ← analogous to $\frac{d}{dx} ax = a$

Gradient Descent

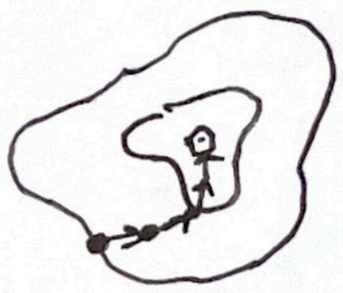
Gradient descent is an iterative method.

We repeat the algorithm multiple towns hopefully stepping toward the desired outcome until ~~the~~ it converges.

1. Starting point.

   Choose a point somewhere in the parameter space. This often chosen randomly.

2. compute the gradient.



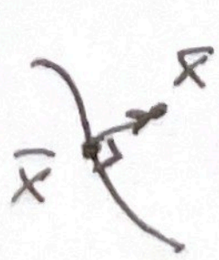
The gradient tells us the direction of steepest ascent.

If we are minimizing then the negative of the gradient tells us the direction of steepest descent.

Aside: In machine learning we are often minimizing a loss and thus we usually refer to this algorithm as gradient descent.

3. Apply an update rule.

   The update takes a step along the gradient.



$\bar{x}'$ ← new location after a step.

~~4. Iterate (go back to 2) until the last update didn't significantly improve.~~

Gradient Descent (cont.)

4. If no improvement
5. Terminate.

We stop iterating when we reach a terminating condition.

5. Iterate (go back to 2)

We reach 5 only if we didn't terminate in step 4.

convergence

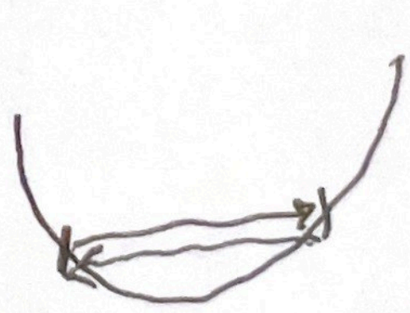
If our surface is concave and we are maximizing, gradient ascent will find the global optimum. provided we use infinitesimal steps.



Gradient descent is guaranteed to converge if the function is convex, and we use dx steps (infinitesimal steps) continuously differentiable

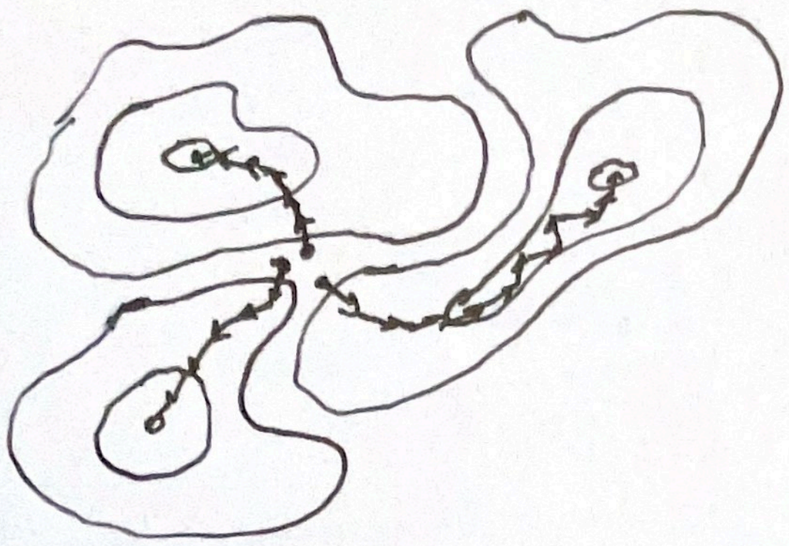


If we use large steps we can oscillate.



If our terminating condition is lack of significant improvement, large steps may prevent convergence by the time the algorithm terminates.

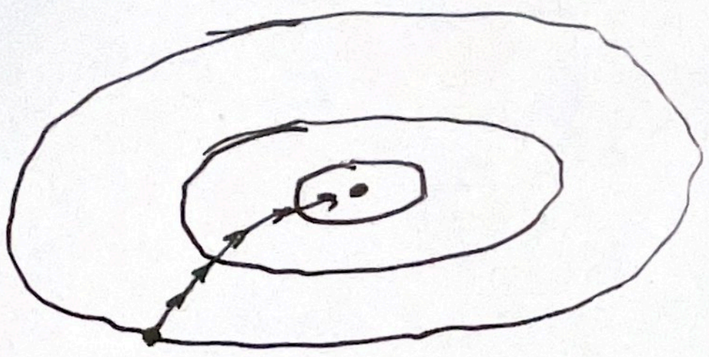
Bad Loss Function



This has multiple local minima.

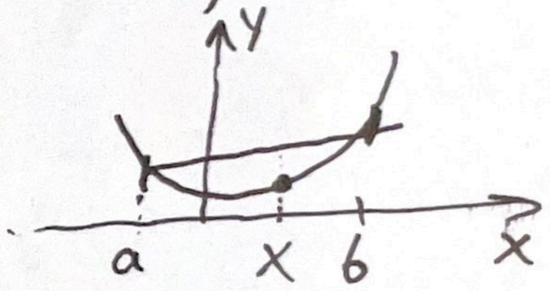
slight change in the starting point can result in a different point of convergence

Good Loss Function



A good loss function has

- a unique minimum
- strictly convex



$$\alpha f(a) + (1-\alpha) f(b) > f(x)$$

for all $a < x < b$.

- continuously differentiable

For any function that is strictly convex and continuously differentiable, $\nabla f(x) = 0$ means x is at the global minimum.

Update functions

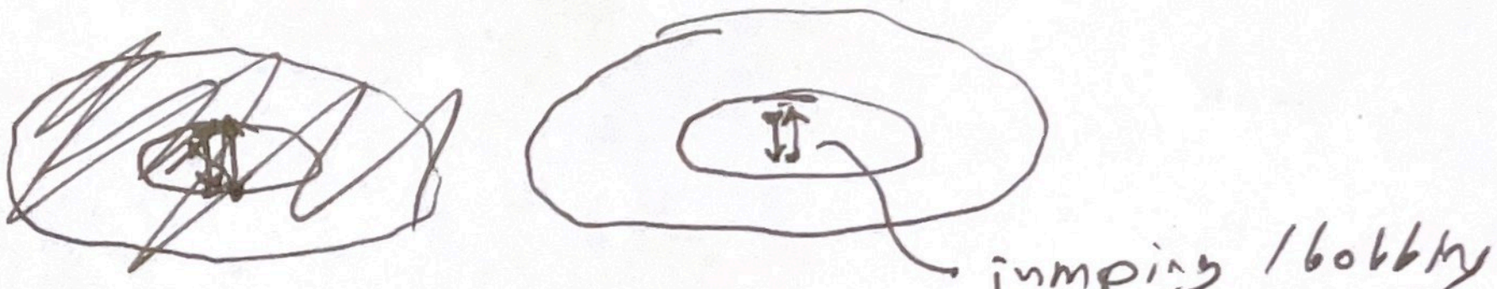
$$\overline{\Theta} := \overline{\Theta} - \alpha \nabla_{\Theta} J(x)$$

- $\overline{\Theta}$: parameters
- $:=$: assignment
- α : learning rate
- $\nabla_{\Theta} J(x)$: gradient with respect to the parameters.
- $J(x)$: cost function

$\alpha < 1$ ← usually $\alpha << 1$ (e.g. 0.001)

Learning Rate

Using a constant learning rate results in bobbing around the optimum.



Depending on the steepness of the loss function, the jumping varies. More steep → more bobbing.

To promote convergence, people often decay the learning rate.

Ex: exponential decay

$$\alpha_t = \alpha_0 e^{-\lambda t}$$

Danger: can stall.

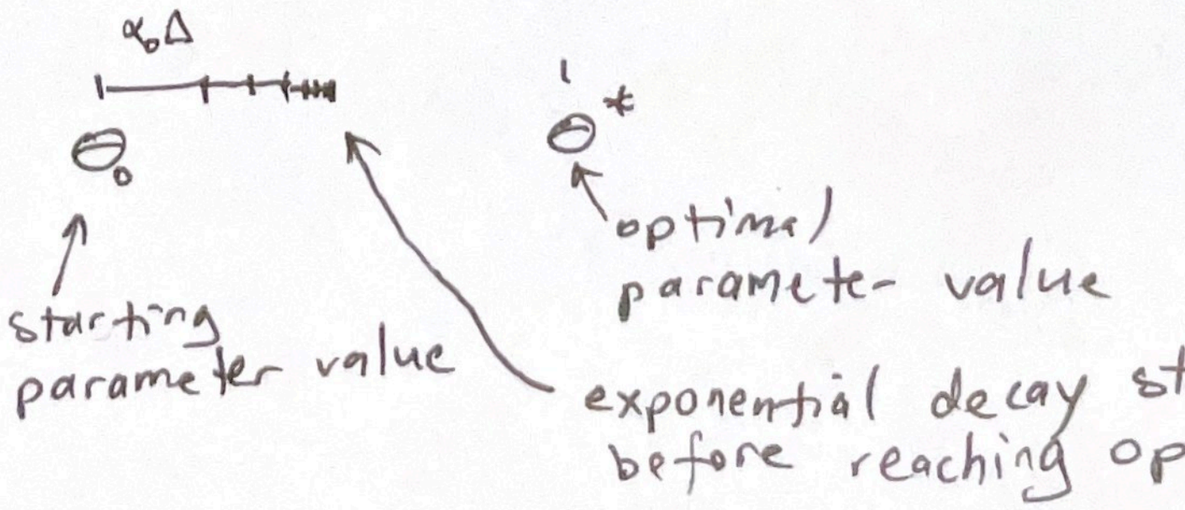
$$\theta_t := \theta_{t-1} + \alpha_0 e^{-\lambda t} \nabla_\theta J$$

assume this consistly points in one direction with magnitude Δ.

$$\theta_t := \theta_{t-1} + \alpha_0 \Delta e^{-\lambda t}$$

$$\theta_t := \theta_0 + \alpha_0 \Delta \underbrace{\sum_{t=0}^{t} e^{-\lambda t}}_{S}, \quad S = 1 + \sum_{t=1}^{\infty} e^{-\lambda t} = 1 + e^{-\lambda} \underbrace{\sum_{t=0}^{\infty} e^{-\lambda t}}_{S}$$

$$S = 1 + e^{-\lambda} S, \quad S = \frac{1}{1 - e^{-\lambda}}$$ This is a constant!



Learning rate (cont.)

Safer learning rate decay: inverse decay

$$\alpha_t = \frac{\alpha_0}{t}$$ ← undefined if t is 0.

$$\alpha_t = \frac{\alpha_0}{1+t}$$ ← At $t=0$, $\alpha_t = \alpha_0$

$$\alpha_t = \frac{\alpha_0}{1+\beta t}$$ ← β allows us to tune how quickly the learning rate decays.

Will inverse decay stall?

$$\theta_t := \theta_{t-1} + \frac{\alpha_0}{1+\beta t} \nabla_\theta J$$ ← Once again assume this points consistently in one direction with magnitude Δ.

$$\theta_t := \theta_{t-1} + \frac{\alpha_0}{1+\beta t} \Delta$$

For $t \gg 0$, the above is approximated by

$$\theta_t := \theta_{t-1} + \frac{\alpha_0}{\beta t} \Delta$$

$$\theta_t \approx \theta_0 + \frac{\alpha_0}{\beta} \Delta \sum_{t=1}^{t} \frac{1}{t}$$ ← S.

$$S = \sum_{t=1}^{\infty} \frac{1}{t} = \infty$$

This means that the sum of the steps never converges to a standstill.

The size of steps decreases so we get closer & closer to the optimum without ever stalling.

Other adaptive learning rates

such as Adam (Adaptive Moment Estimation).

Adapts based on past gradients

Logistic Regression

Logistic Regression is a discriminative algorithm

Ganssian Discriminant Analysis is a generative algorithm

$$P(y|\bar{x}) = \frac{P(\bar{x}|y)\,P(y)}{P(\bar{x})}$$

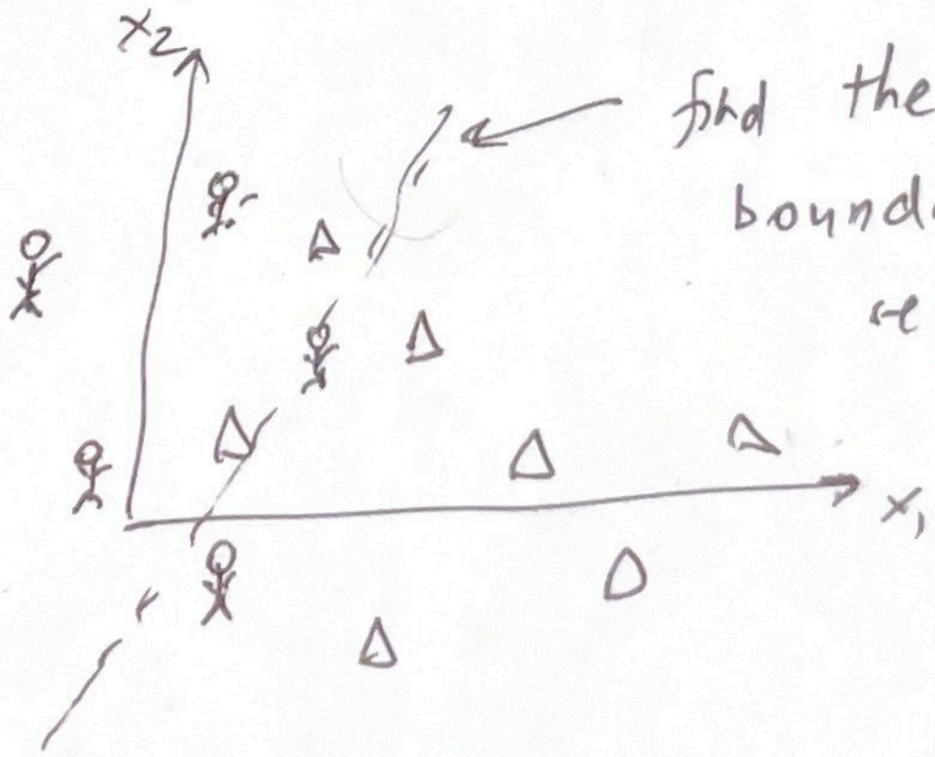
This is what we really want.

with a generative algorithm we model the likelihood of $\bar{x}$ given y, and we model the prior probabilities of the classes.

with a discriminative model, model the posterior directly.

$P(y|\bar{x})$

↖ model this!



← find the linear decision boundary that best separates the classes

Logistic Regression

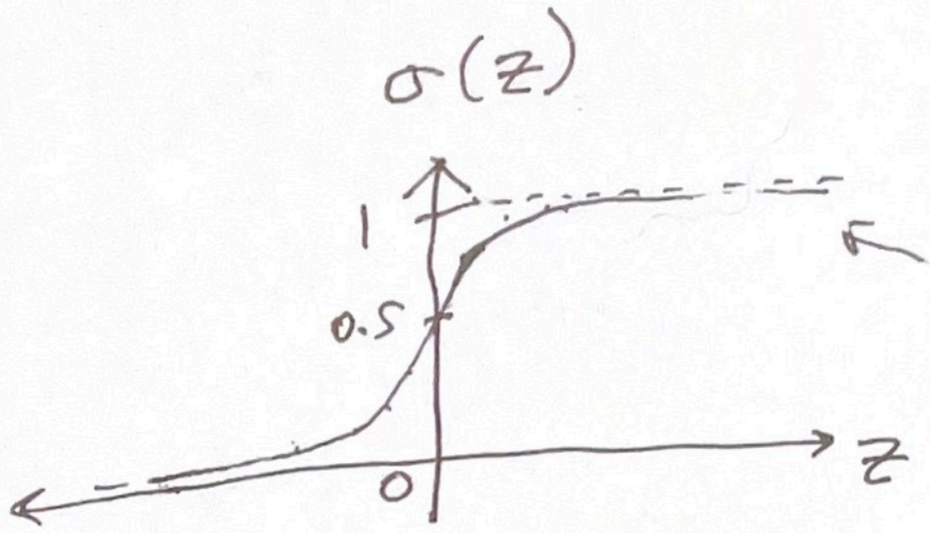
$h_\theta(\bar{x}) \in [0,1]$

$\hat{y} = h_\theta(\bar{x}) = \sigma(\theta^T \bar{x}) = \frac{1}{1+e^{-\theta^T \bar{x}}}$ } logistic function

$\sigma \triangleq$ sigmoid

S-shaped curves are called sigmoids

σ = logistic function is a type of sigmoid.



↖ supposed to be symmetric about $\sigma(0) = 0.5$.

~~$\sigma(z) = 0.5$~~

Let $P(y=1 \mid \bar{x}; \theta) = h_\theta(\bar{x})$

So $\hat{y} = P(y=1 \mid \bar{x}; \theta)$ means the output represents the predicted posterior probability that $y=1$.

Another way to look at $\theta^T \bar{x}$

unnormalized

Think of $\theta^T \bar{x}$ as the signed distance from the plane defined by $0 = \theta^T \bar{x}$.

The hyperplane of the decision boundary is

$0 = \theta^T \bar{x}$ where $\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$, $\bar{x} = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$

No 1.

This can be rewritten as

$0 = \bar{w}^T \bar{x} + b$ where $\bar{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_n \end{bmatrix}$, $\bar{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$

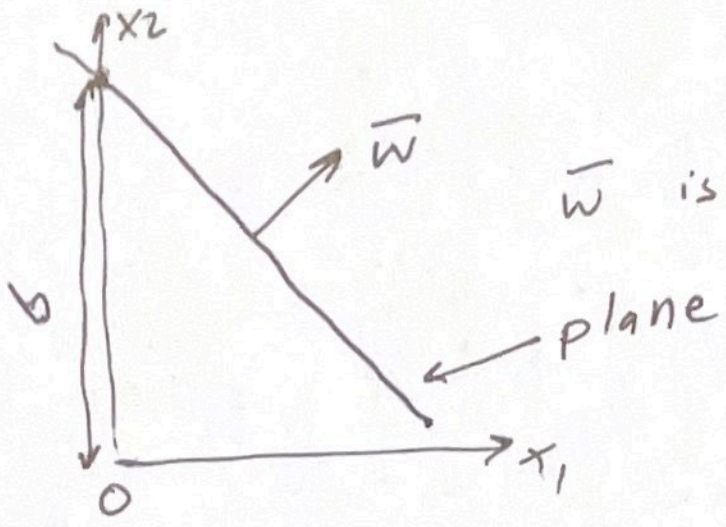
↖ no w_0.

↑ weights ↖ bias

Logistic Regression (cont.)

In homework we showed $(\theta_1, \cdots, \theta_n)$ is a surface normal to the plane $0 = \theta^T \bar{x}$. ← $\theta^T \bar{x}$

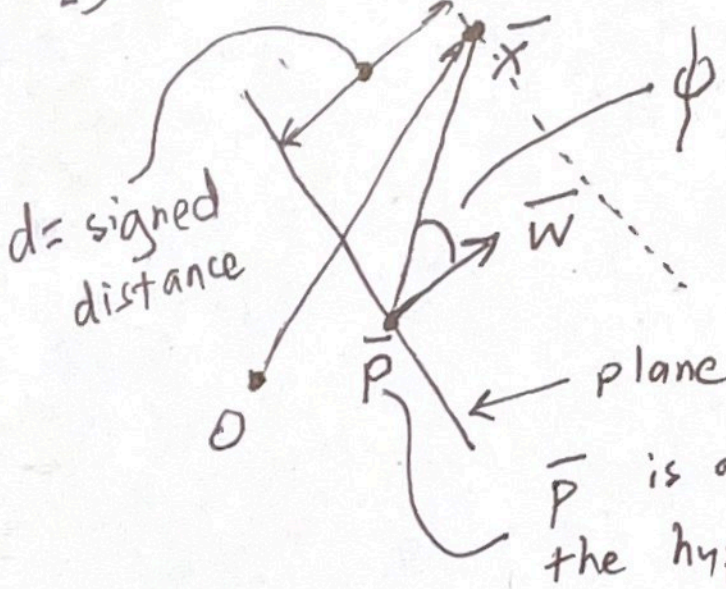
When represented as $0 = \bar{w}^T \bar{x} + b$, the surface normal is $(w_1, \cdots, w_n)$.



$\bar{w}$ is a surface normal vector.

Only when $\bar{x}$ is on the plane, $0 = \bar{w}^T \bar{x} + b$.

If $\bar{x}$ is not on the plane?



$$\bar{w}^T(\bar{x} - \bar{p}) = \|\bar{w}\| \underbrace{\|\bar{x} - \bar{p}\| \cos\phi}_{d = \text{signed distance from } \bar{x} \text{ to the plane.}} \quad (1)$$

$$d = \frac{\bar{w}^T \bar{x} - \bar{w}^T \bar{p}}{\|\bar{w}\|} \quad (2)$$

$\bar{p}$ is on the plane and thus $\bar{w}^T \bar{p} + b = 0$, $b = -\bar{w}^T \bar{p}$. (3)

Substituting (3) into (2) yields ← unit normal

$$d = \frac{\bar{w}^T \bar{x} + b}{\|\bar{w}\|} = \frac{\bar{w}^T \bar{x}}{\|w\|} + \frac{b}{\|\bar{w}\|} = \hat{w}^T \bar{x} + \frac{b}{\|\bar{w}\|} \quad (4)$$

We can think of $\bar{w}^T \bar{x} + b$ as the weighted signed distance from the plane to $\bar{x}$. From (4)

$$\|w\| d = \bar{w}^T \bar{x} + b \quad (5)$$

$\|w\|$: weighted by the length of the normal vector defined by the weight parameters of $h_{\bar{w},b}(\bar{x})$.

d: signed distance

$$h_{\bar{w},b}(\bar{x}) = \sigma(\underbrace{\bar{w}^T \bar{x} + b}_{z}) = \sigma(\underbrace{\|w\| d}_{z}) \quad (6)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Logistic Regression (CONT.)

How do we set $\bar{\theta}$ (or equivalently $\bar{w}$ & b) to position the plane on a good decision boundary between classes?

- Logistic Regression is a <u>discriminative method</u>. It directly models the posterior class probabilities $P(y|x;\theta)$ and learns parameters θ that maximize the conditional likelihood of the observed labels.
  - we model the distribution $P(y=\omega_i|x;\theta)$

- Gaussian Discrimant Analysis (GDA) in contrast is a <u>generative method</u>. It models how the features $\bar{x}$ are "generated" for each class. It models $P(x|y)$ and class prior(s) $P(y)$. Using Bayes' rule, we estimate the posterior class probabilities as

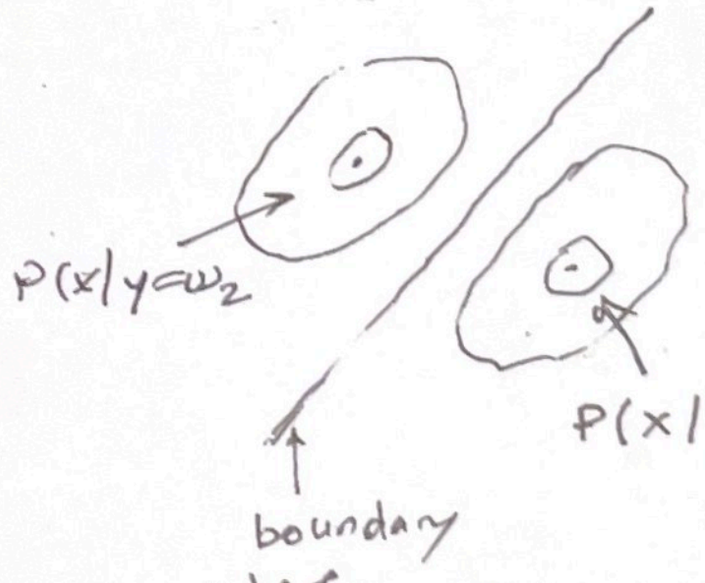
$$P(y|x) \propto \underbrace{P(x|y)}_{\text{we model these separately for each class}} P(y)$$

$P(x|y=\omega_1)$ $P(y=\omega_1)$

$P(x|y=\omega_2)$ $P(y=\omega_2)$

$\vdots$

$P(x|y=\omega_n)$ $P(y=\omega_3)$



Aside: "discriminant" is an odd choice of names given that it is so close to "discriminative." "Discriminant" refers to the decision rule, but let's not go into that here. Just remember the naming oddity and that GDA is generative.

Table 1: Summary

Approach	Models directly	Examples
Discriminative	$P(y\|x)$	Logistic Regression
Generative	$P(x\|y)$ and $P(y)$	GDA, QDA,

Logistic Regression (cont.)

So how do model $P(y|x)$ directly?

1) Treat the training data as givens (constants)

while optimize Θ, the model parameters.

$P(y|x;\Theta)$ — Θ: varied; x: held constant

2) we maximize the "probabilities of the correct labels."

$$\max_{\Theta} \prod_{i=1}^{m} P(y^{(i)} | x^{(i)}; \Theta) \quad (8)$$

"Likelihood of the class labels"

Why is this called a likelihood? I thought

$$p(y|x) = \frac{p(x|y)\,p(y)}{p(x)}$$

$p(y|x)$: posterior probability of the class

$p(x|y)$: likelihood of x

$p(y)$: class prior

The more formal name for the likelihood of x is "class-conditional density" or just referred to using plain English as "likelihood of the features given the class."

This is often a point of confusion. "Likelihood" without additional qualifiers is inherently ambiguous.

What distinguishes "likelihood of the class labels" from "posterior probability of the class labels"?

It's all about what is treated as a variable.

Likelihood of the class labels

- Hold the training data as a constant
  $\{(x^{(i)}, y^{(i)})\}$ where $i = 1, \ldots, m$

- Consider the probability of the observed labels a function of <u>only Θ</u>.

$$L(\Theta; \text{data}) = \prod_{i=1}^{m} P_\Theta(y^{(i)} \mid x^{(i)}) \quad (9)$$

"likelihood function" or $L(\Theta; \text{data}) = \prod_{i=1}^{m} P(y^{(i)} \mid x^{(i)}; \Theta)$

- Variable is Θ.

we put Θ here to emphasize that we "treat this as a function of Θ."

Posterior probability of the class labels

- Fix Θ and the data.
- compute the probability of the class labels.

$$P_\Theta(y \mid x)$$

- Both Θ and data are fixed (constant)

Likelihood assuming IID sampling

Let $D = \{(x^{(i)}, y^{(i)})\}$ for $i = 1, \ldots, m$.

The most general way to write the likelihood function L is

$$L(\theta; D) = P_\theta(D)$$ ← This is the probability of the entire training data set.

As long as we avoid bias in our sampling, i.e., as long as our sampling is iid (independent and identically distributed) when building the training dataset

$$L(\theta; D) = P_\theta(y^{(1)}|x^{(1)}) P_\theta(y^{(2)}|x^{(2)}) \ldots$$

$$= \prod_{i=1}^{m} P_\theta(y^{(i)}|x^{(i)}) \quad (10)$$

Maximum Likelihood Estimation (MLE)

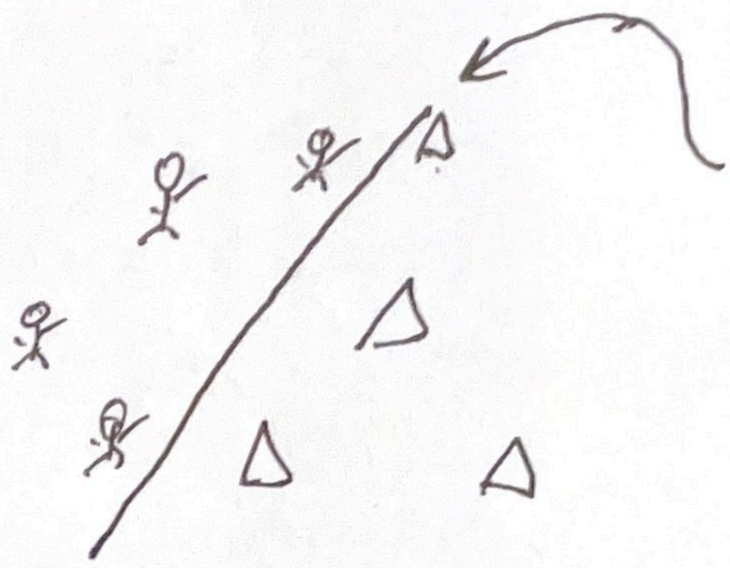
MLE is finding the θ that maximizes the likelihood function

$$\theta_{MLE} = \underset{\theta}{\operatorname{argmax}} \; L(\theta; D)$$

$$= \underset{\theta}{\operatorname{argmax}} \prod_{i=1}^{m} P_\theta(y^{(i)}|x^{(i)}) \quad (11)$$

How does MLE relate to logistic regression? 18

Logistic regression finds Θ_{MLE} assuming a model with a decision boundary that is a hyperplane.



with logistic regression this decision boundary has Θ_{MLE}.

This is an oversimplification because how we set Θ_{MLE} depends on more than decision boundary. It depends on how we model $p(y|x)$.

Ideally, our model $h_\Theta(\bar{x})$ exactly produces the posterior probability for all classes. (12)

$$h_\Theta(\bar{x}) = p(y|\bar{x};\Theta)$$

Lets limit ourselves to binary classification.

We can thus rewrite (11) as (13)

$$p(y=1|\bar{x};\Theta) = h_\Theta(\bar{x})$$

$$p(y=0|\bar{x};\Theta) = 1 - p(y=1|\bar{x};\Theta) = 1 - h_\Theta(\bar{x}) \quad (14)$$

We can combine (14) and (13) into a single equation (15)

$$p(y|\bar{x};\Theta) = h_\Theta(\bar{x})^y (1-h_\Theta(\bar{x}))^{1-y}$$

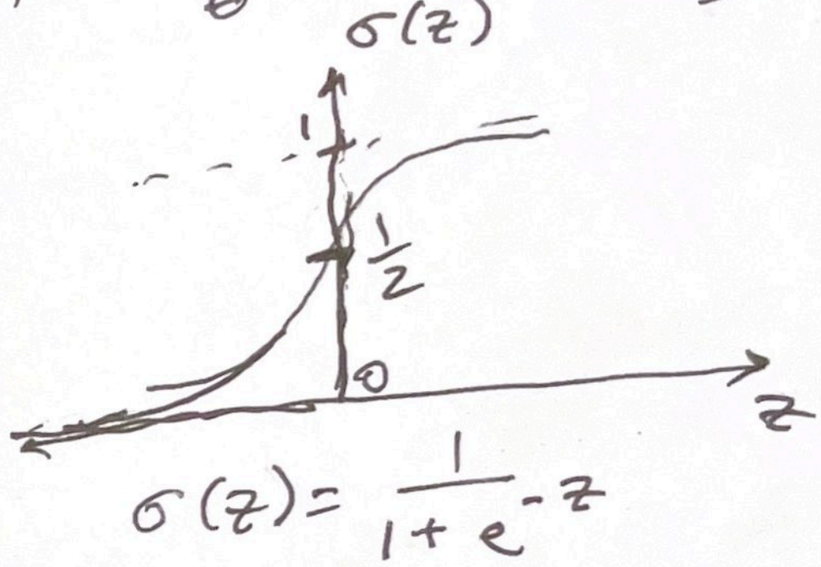
when $y=1$, $h_\Theta(x)^1 = h_\Theta(x)$ but $(1-h_\Theta(\bar{x}))^{1-1} = 1$. y acts like a switch.

Using Maximum Likelihood Estimation (MLE), we combine (10) and (15) to get

$$\Theta_{MLE} = \underset{\Theta}{\operatorname{argmax}} \prod_{i=1}^{m} h_\Theta(\bar{x}^{(i)})^{y^{(i)}} (1 - h_\Theta(\bar{x}^{(i)}))^{1-y^{(i)}} \quad (16)$$

With logistic regression we constrain $h_\Theta(\bar{x})$ to being a logistic function

$$h_\Theta(x) = \sigma(\Theta^T \bar{x})$$



The decision boundary occurs at

$$h_\Theta(\bar{x}) = P(y=1 \mid \bar{x}; \Theta) = \frac{1}{2}$$ ← 50% probability of either class.

The maxima Θ_{MLE} will occur where

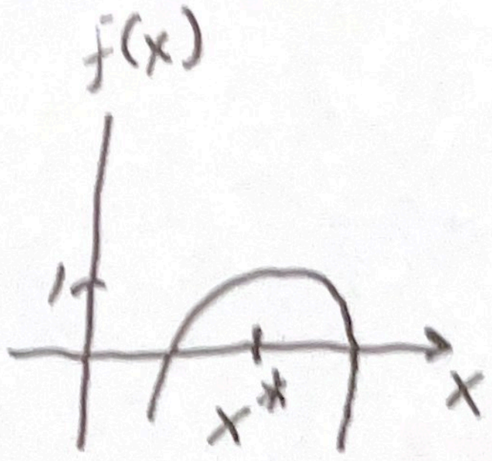
$$\nabla_\Theta L(\Theta; D) = 0 \quad (17)$$

Substituting (10) and (15) into (17) yields

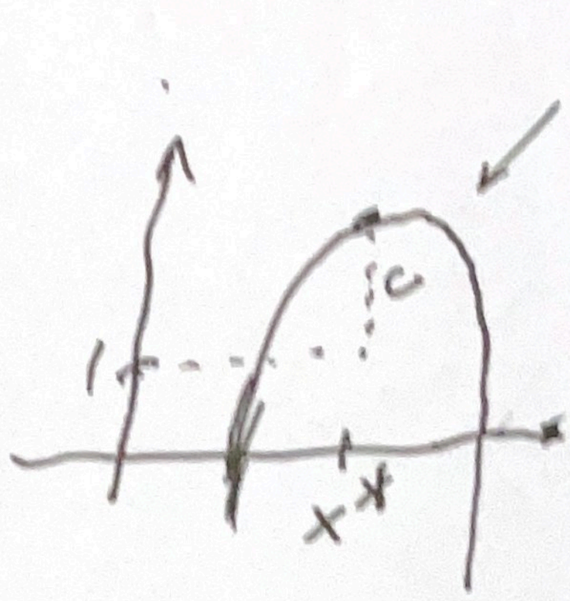
$$\nabla_\Theta \prod_{i=1}^{m} h_\Theta(\bar{x}^{(i)})^{y^{(i)}} (1 - h_\Theta(\bar{x}^{(i)}))^{1-y^{(i)}} = 0 \quad (18)$$

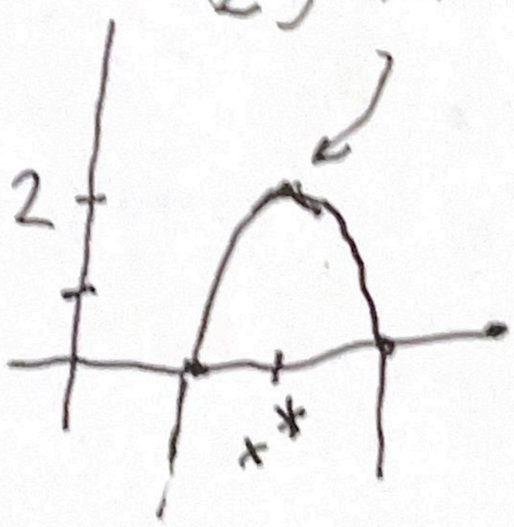
It is much easier to deal with summations than products. Can I transform (18) into a different problem that has a zero gradient at the same Θ?

When does transforming a problem preserve the argmax?

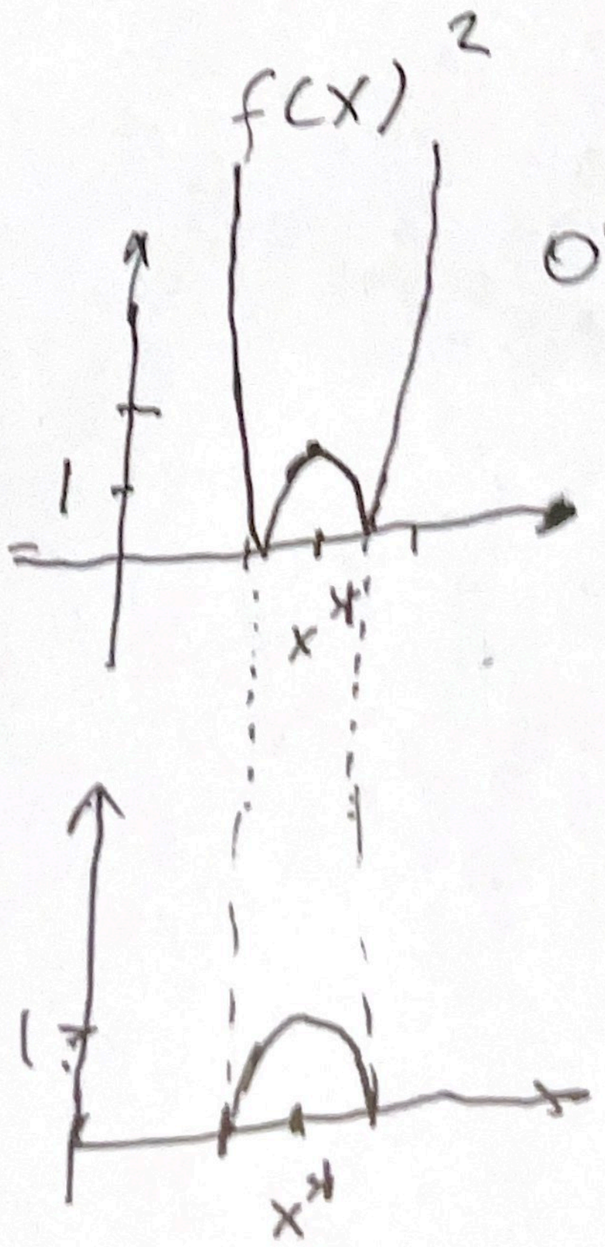


+ C





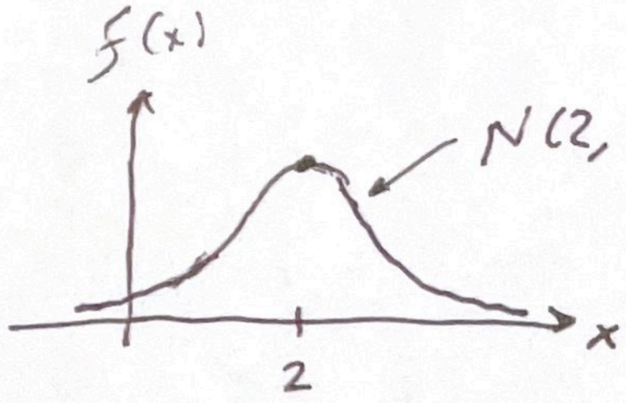
or more generally multiplying f(x) by any positive constant



oops! Mutiplying a function by itself resulted in changing the max to ∞.

But squaring works as long as we limit our domain to where f(x) ≥ 0.

Further examples of transforming without moving the argmax.



$$N(2,1) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-2)^2}{2}}$$

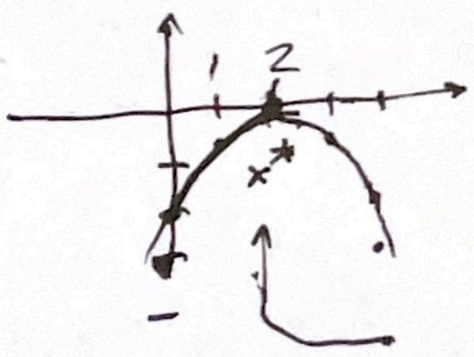
$$\operatorname*{argmax}_x N(2,1) = 2$$

$$\operatorname*{argmax}_x \log N(3,1) = \operatorname*{argmax}_x \log \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-2)^2}{2}}$$

$$= \operatorname*{argmax}_x \left(\log \frac{1}{\sqrt{2\pi}} + \log e^{-\frac{(x-2)^2}{2}} \right)$$

$$= \operatorname*{argmax}_x \left(\log \frac{1}{\sqrt{2\pi}} - \frac{(x-2)^2}{2} \right)$$

↑ adding a constant to a function that is always ≥ 0 does not move argmax so



$$= \operatorname*{argmax}_x -\frac{(x-2)^2}{2} = 2$$

More generally

$$\operatorname*{argmax}_x f(x) = \operatorname*{argmax}_x g(f(x))$$

in the domain where $f(x) \geq 0$

and g is strictly increasing

This is called a <u>monotone transform</u>.

$\log(x)$ is monotonically increasing for $x > 0$.

$$\therefore \operatorname*{argmax}_{*\theta} L(*\theta; D) = \operatorname*{argmax}_{\theta} \log L(\theta; D) \quad (19)$$

Define $l = \log L$ we restate our problem in (19) as

$$\theta_{MLE} = \underset{\theta}{\operatorname{argmax}}\ l(\theta; D) \tag{20}$$

where $l(\theta; D) = \log \prod_{i=1}^{m} h_\theta(\bar{x}^{(i)})^{y^{(i)}} (1 - h_\theta(\bar{x}^{(i)}))^{1-y^{(i)}}$

$$= \sum_{i=1}^{m} \left[y^{(i)} \log h_\theta(\bar{x}^{(i)}) + (1-y^{(i)}) \log (1 - h_\theta(\bar{x}^{(i)})) \right] \tag{21}$$

where h_θ is a logistic function given by $\sigma(\theta^T \bar{x}^{(i)})$.

The argmax occurs where the gradient is zero as in

$$\nabla_\theta\ l(\theta; D) = 0 = \begin{bmatrix} \frac{\partial}{\partial \theta_0} l(\theta; D) \\ \frac{\partial}{\partial \theta_1} l(\theta; D) \\ \vdots \\ \frac{\partial}{\partial \theta_n} l(\theta; D) \end{bmatrix} \tag{22}$$

This is a zero vector.

For a single dimension of the gradient vector, we combine (21) & (22)

$$\frac{\partial}{\partial \theta_j} \sum_{i=1}^{m} \left[y^{(i)} \log \sigma(\theta^T \bar{x}^{(i)}) + (1-y^{(i)}) \log (1 - \sigma(\theta^T \bar{x}^{(i)})) \right] = 0 \tag{23}$$

Before continuing from (23) let's consider the derivative of the logistic function.

$$\frac{d}{dz} \sigma(z) = \frac{d}{dz} \frac{1}{1+e^{-z}} = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{1-1+e^{-z}}{1+e^{-z}}$$

$$= \frac{1}{1+e^{-z}} \cdot \left(1 - \frac{1}{1+e^{-z}}\right) = \sigma(z)(1-\sigma(z)) \tag{24}$$

[ogistic regression (cont.)

Compute the derivative of (23)

$$\sum_{i=1}^{m} y^{(i)} \underbrace{\frac{\partial}{\partial \theta_j}\left[\log \sigma(\theta^T \bar{x}^{(i)})\right]}_{①} + \sum_{i=1}^{m}(1-y^{(i)}) \underbrace{\frac{\partial}{\partial \theta_j}\left[\log\left(1-\sigma(\theta^T \bar{x}^{(i)})\right)\right]}_{②} = 0 \quad (24)$$

From (24) ①, we get

$$\frac{\partial}{\partial \theta_j}\left[\log \sigma(\theta^T \bar{x}^{(i)})\right] = \frac{1}{\sigma(\theta^T \bar{x}^{(i)})} \underbrace{\frac{\partial}{\partial \theta_j} \sigma(\theta^T \bar{x}^{(i)})}_{\sigma(z)(1-\sigma(z)) \text{ where } z = \theta^T \bar{x}^{(i)}}$$

$$= \frac{1}{\sigma(\theta^T \bar{x}^{(i)})} \sigma(\theta^T \bar{x}^{(i)})\left(1-\sigma(\theta^T \bar{x}^{(i)})\right) \frac{\partial}{\partial \theta_j}(\theta^T \bar{x}^{(i)})$$

$$= \left(1-\sigma(\theta^T \bar{x}^{(i)})\right) \bar{x}_j^{(i)} \quad (25)$$

should just be $x_j^{(i)}$

From (24) ②, we get and let $z = \theta^T \bar{x}^{(i)}$, we get

$$\frac{d}{dz}\log(1-\sigma(z)) = \frac{1}{1-\sigma(z)} \frac{d}{dz}(1-\sigma(z)) = \frac{1}{1-\sigma(z)} \cdot -\frac{d}{dz}\sigma(z)$$

$$= -\frac{1}{1-\sigma(z)} \cdot \sigma(z)(1-\sigma(z)) = -\sigma(z)$$

$$\frac{\partial}{\partial \theta_j}\log\left(1-\sigma(\theta^T \bar{x}^{(i)})\right) = -\sigma(z) \frac{\partial}{\partial \theta_j}(\theta^T \bar{x}^{(i)}) = -\sigma(\theta^T \bar{x}^{(i)})\, x_j^{(i)} \quad (26)$$

Substituting (25) & (26) into (24) yields

$$\frac{\partial}{\partial \theta_j} l(\theta; D) = \sum_{i=1}^{m}\left[y^{(i)}\left(1-\sigma(\theta^T \bar{x}^{(i)})\right) + (1-y^{(i)}) \cdot -\sigma(\theta^T \bar{x}^{(i)})\right] x_j^{(i)}$$

$$= \sum_{i=1}^{m}\left[y^{(i)} - \sigma(\theta^T \bar{x}^{(i)})\right] x_j^{(i)} \quad (27)$$

logistic regression (cont.)

combining (22) and (27) yields

$$\nabla_\theta \ell(\theta; D) = \begin{bmatrix} \sum_{i=1}^{m} \Delta^{(i)} \cdot 1 \\ \sum_{i=1}^{m} \Delta^{(i)} x_1^{(i)} \\ \vdots \\ \sum_{i=1}^{m} \Delta^{(i)} x_n^{(i)} \end{bmatrix} \quad \text{where } \Delta^{(i)} = y^{(i)} - \sigma(\theta^T \bar{x}^{(i)}) = y^{(i)} - \hat{y}^{(i)} \tag{28}$$

When the gradient is 0, θ is at a maxima.

When gradient is not 0, we step in the direction of the maxima toward the maxima.

We can define an update function

$$\theta := \theta + \alpha \nabla_\theta \ell(\theta; D) \tag{29}$$

Because we are maximizing the log likehood function, this is an example of gradient ascent.

In ML we usually talk in terms of ~~grad~~ loss functions and minimizing cost. To turn (29) into gradient descent, we define the cost function as

$$J(\theta; D) = -\ell(\theta; D)$$

and (29) then becomes

$$\theta := \theta - \alpha \nabla_\theta J(\theta; D) \tag{30}$$

where $\frac{\partial J}{\partial \theta_j} = \sum_{i=1}^{m} (\sigma(\theta^T \bar{x}^{(i)}) - y^{(i)}) x_j^{(i)}$

logistic regression (cont.)

Instead of using $\theta^T \bar{x}$ $\left(\begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}\right)$ we can reformulate the update function using $\bar{w}^T \bar{x} + b$ where $\bar{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$ we removed the one.

Our hypothesis becomes

$$h_{\bar{w},b}(\bar{x}) = \sigma(\bar{w}^T \bar{x} + b) = \sigma(\|w\| d)$$

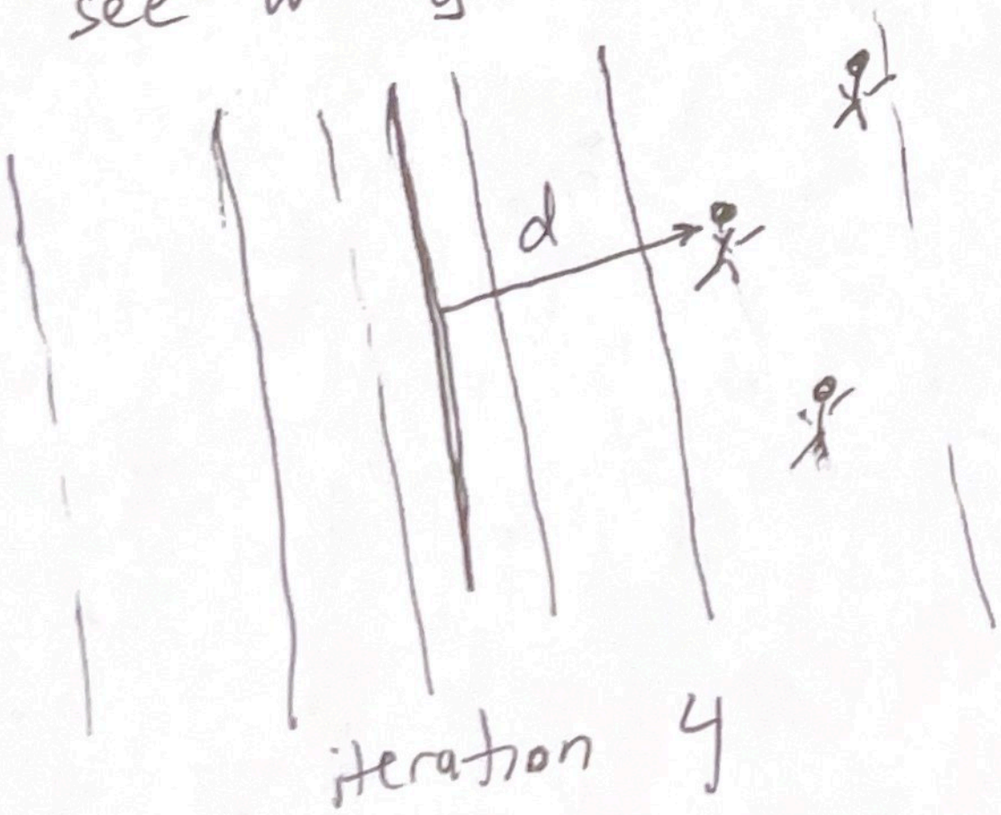
This is identical to (6) on p 14.

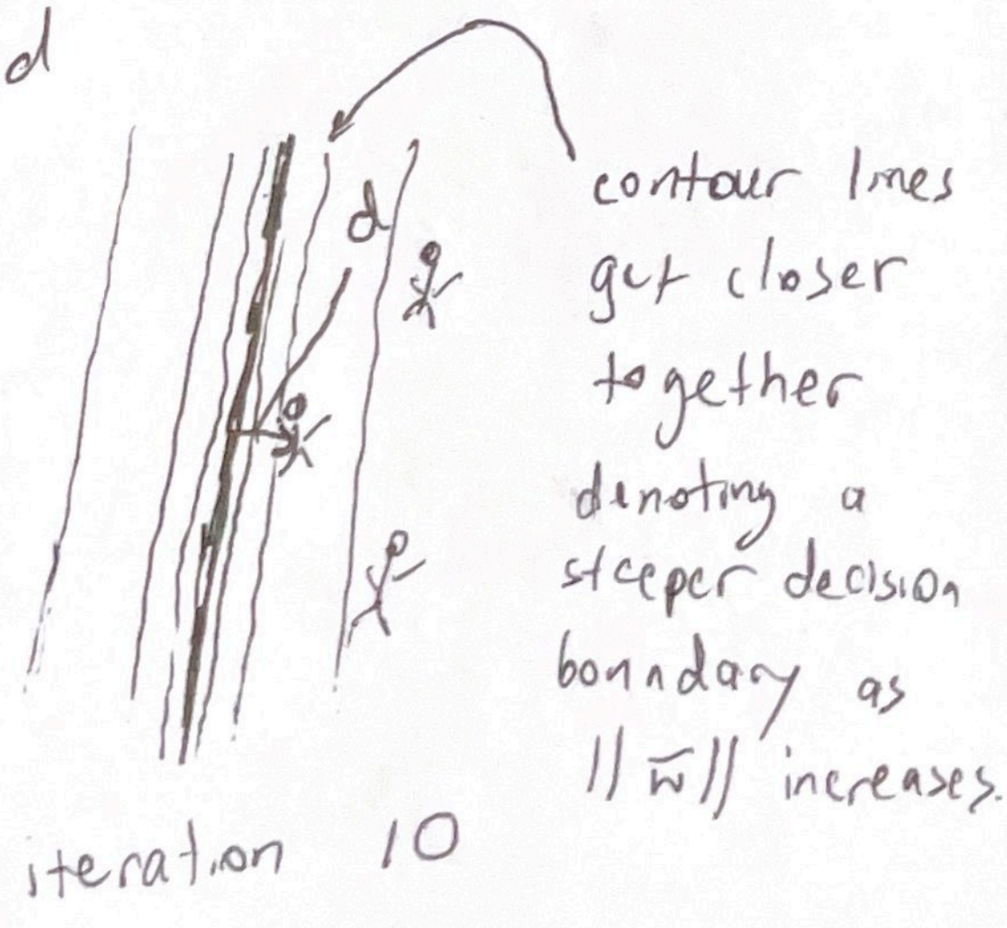
The update rule thus becomes

$$w_j := w_j - \alpha \sum_{i=1}^{m} \left(\sigma(\|\bar{w}\| d^{(i)}) - y^{(i)}\right) x_j^{(i)}$$

$$b := b - \alpha \sum_{i=1}^{m} \left(\sigma(\|\bar{w}\| d^{(i)}) - y^{(i)}\right)$$

With each update, more points in the data set end up on the correct side of the decision boundary until near the optimal where the decision boundary wobbles. When $y^{(i)} = 1$ and $\bar{x}^{(i)}$ is on the correct side of the decision boundary $d^{(i)}$ is small but positive cause $\bar{w}$ to experience a small nudge toward $\bar{x}^{(i)}$. When the two classes are linearly separable or mostly linearly separable we see $\bar{w}$ grow without bound





logistic regression (cont.)

Because (30) updates parameters once per pass over all the data D in the training data set, this is called batch gradient descent.

Stochastic Gradient Descent

Instead of updating the parameter once per pass over the training data D, with stochastic gradient descent, we ~~adjust~~ update the parameters for each sample in the training data D.

update rule

$$\theta_j := \theta_j - \alpha\left(\sigma(\theta^T \bar{x}^{(i)}) - y^{(i)}\right) x_j^{(i)}$$

For SGD to be effective, we choose a much smaller value for α than we would for batch gradient descent. If α is too large, the decision boundary jump around the optimal with slow convergence.

SGD tends to converge faster than Batch Gradient Descent (BGD) especially for larger training data sets. Because SGD bounces around a little with each update, it can sometimes work around shallow local minima sometimes encountered with deep learning.